Invoice Generator — Project Documentation

Oct 12, 2026 · @Trula Ai

Overview

Invoice Generator is a multi-tenant billing web app for small businesses in India: it issues GST-compliant invoices, quotes and credit notes, records or collects payments, and chases unpaid invoices by email on a fixed schedule. Every business is an isolated tenant with its own users, clients, documents and settings.

Status on 12 Oct 2026: feature-complete first version. Source lives on GitHub at trula-ai/InvoiceGenerator; the app is deployed on Render with a hosted Supabase PostgreSQL database and sends email through Hostinger SMTP. Earlier documentation (6 Oct 2026, PDF in the repo's docs folder) predates quotes, credit notes, the item catalog, recurring invoices, Razorpay checkout, the reminder schedule and the owner notify mail; this document supersedes it.

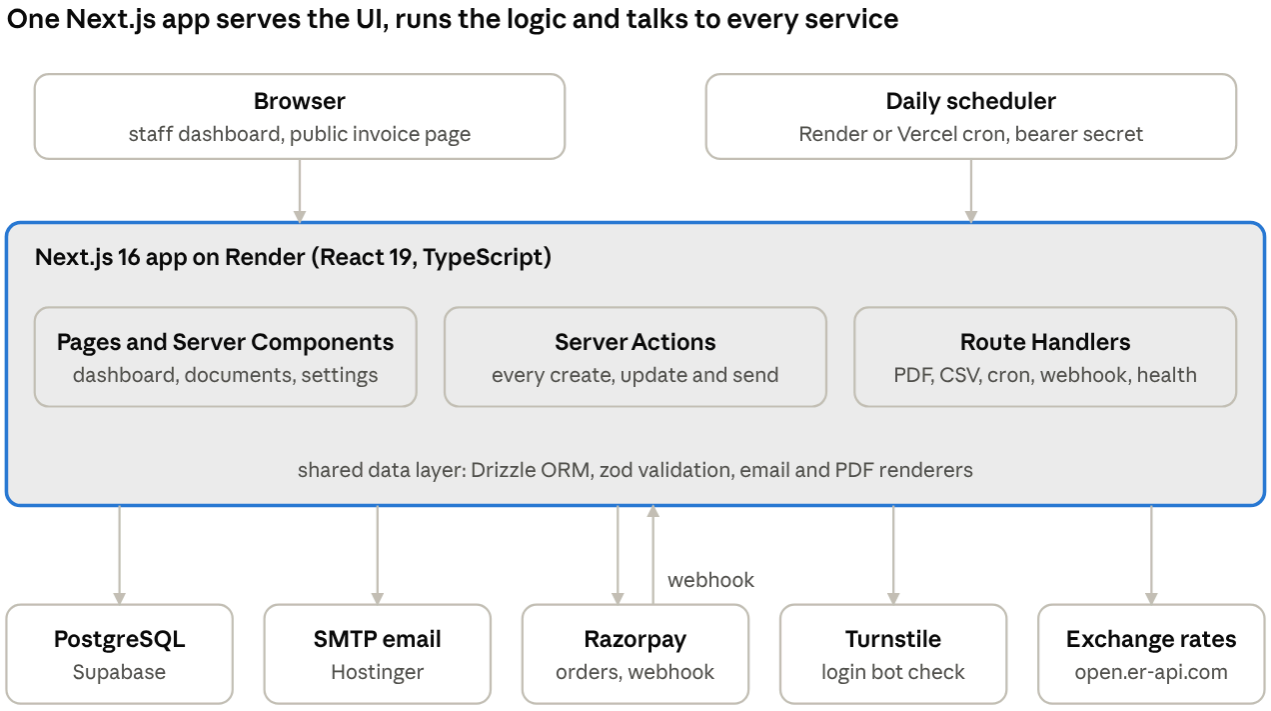
Tech stack

A single Next.js application serves the UI, the server logic and the API routes; there is no separate backend service.

Layer	Choice	Notes
Framework	Next.js 16.3 (App Router), React 19.2, TypeScript 5	Server Components, Server Actions for every mutation, Route Handlers for PDFs, CSV, cron and webhooks; Turbopack dev and build
UI	Tailwind CSS 4, shadcn/ui on Base UI, lucide-react icons, sonner toasts	Neumorphic card style, 7 accent themes plus light/dark/system via next-themes
Forms and validation	react-hook-form 7, zod 4	The same zod schema validates in the browser and inside each Server Action
Database	PostgreSQL 17 on Supabase (production), local PostgreSQL (development)	Drizzle ORM 0.45 with drizzle-kit migrations (8 applied); postgres.js driver with prepared statements off for pooler compatibility

Layer	Choice	Notes
Money	decimal.js, numeric(14,2) columns	No floats; INR is the reporting currency, exchange rates frozen per document from open.er-api.com
PDF	@react-pdf/renderer 4	Invoice, quote, credit note and receipt documents rendered on the server
Email	nodemailer 7 over SMTP (Hostinger), Resend 6 as alternative, console fallback	Templates built with @react-email/components
Online payments	Razorpay REST API and hosted Checkout	No SDK bundled; signature verification and webhook on the server
Auth and security	scrypt password hashing, database-backed sessions, Cloudflare Turnstile	httpOnly session cookie, single-use password-reset tokens
Dates	date-fns 4, react-day-picker 10	Indian financial-year numbering (April to March)
Hosting	Render web service; GitHub trula-ai/InvoiceGenerator	vercel.json cron config also present, so Vercel works too
Tooling	ESLint 9 with eslint-config-next, drizzle studio	npm scripts: dev, build, start, lint, db:generate, db:migrate, db:push, db:studio

Architecture



architecture · one app, five external services

The browser and the daily scheduler both hit the same Next.js app: pages render on the server, every change goes through a Server Action that validates with zod and writes through Drizzle, and Route Handlers serve PDFs, CSV exports, the cron job, the Razorpay webhook and the health check. Folders follow that split: app (routes), components (UI), lib/actions, lib/data, lib/email, lib/pdf, lib/payments, db/schema and db/migrations.

Authentication and security

Every request is tied to one business: the signed-in user's business id scopes every query and Server Action, which is what isolates tenants.

- **Register** creates a business and its first user in one transaction, then signs them in and lands on Settings.
- **Login** verifies a Cloudflare Turnstile token before any database work, then checks the scrypt password hash. Failed attempts show a generic message and request a fresh challenge.
- **Sessions** are rows in the database referenced by a random 256-bit token in an httpOnly, SameSite=Lax cookie. A lightweight proxy redirects unauthenticated visitors away from the dashboard before rendering.

- **Forgot password** emails a single-use reset link valid for 60 minutes. The response is identical whether or not the address exists. Resetting the password signs the user out everywhere and starts a fresh session.
- **Account settings** let a user change their name, email and password.
- **Error hygiene**: database and driver errors are logged on the server and replaced with a plain message, so SQL never reaches the screen.

Clients

Clients are the people and companies you bill; each carries the details the GST rules and the emails need.

- B2B or B2C type, name, contact person, email, phone and GSTIN; the type sets the default invoice type.
- Billing address with an Indian state code, which decides CGST+SGST versus IGST on each invoice.
- Preferred invoicing currency, internal notes, and an **Automatic reminders** switch to exclude a client from scheduled reminder emails (manual sends still work).
- List page with type-ahead search across name, contact, email, phone and GSTIN, type filter, archived toggle and CSV export. Every row ends with a **Send payment reminder** button.
- Client page shows invoiced and outstanding totals, the full invoice history with a per-invoice reminder button and the last-reminded date, plus archive (soft delete that keeps history).

Invoices, quotes and credit notes

All three are one document type with a kind flag, so they share line items, tax logic, PDFs and the public link while keeping separate numbering.

- **Numbering** per financial year with configurable prefixes, e.g. INV/2026-27/0001, allocated by an atomic counter so two users can never get the same number.
- **Line items** picked from the item catalog or typed in: description, HSN/SAC code, quantity, unit, price and tax rate; catalog edits never change issued documents.
- **GST**: the business's GST setting, GSTIN, place of supply and inter-state decision are snapshotted on the document; CGST/SGST or IGST are computed per line with an HSN summary on the PDF. GST can be switched off for plain per-item tax.
- **Totals**: discount (none, percent or fixed) allocated across lines, optional round-off, amount in words, and INR equivalents frozen with the exchange rate at issue time for any currency (INR, USD, EUR, GBP, AED and more).

- **Extras on the document**: PO number, ship-to address, notes, terms, bank details, logo, and an authorised-signatory block with signature image.
- **Lifecycle**: draft, pending, partially paid, paid, overdue (set automatically past the due date), cancelled. Drafts can be edited or deleted; unpaid invoices can be edited; any document can be duplicated.
- **Quotes** carry a validity period, can be accepted or declined by the client from the public link, and convert into an invoice in one click. **Credit notes** record reductions against invoices.
- **Recurring profiles** are created from an issued invoice with a frequency from weekly to yearly, can be paused, ended or run on demand, and generate the next invoice automatically during the daily checks.
- **Delivery**: PDF preview and download, email with PDF attached, a copyable public link (/i/<token>) where the client can view, download, accept a quote or pay online with no login.

Payments, receipts and online payments

A payment can be recorded by hand or collected online through Razorpay; either way the invoice balance, status and receipt are updated in one transaction.

- **Record payment** from the invoice page: amount, date, method (bank transfer, UPI, card, cash, cheque, other), reference and notes. Partial payments are allowed; the amount can never exceed the outstanding balance.
- **Receipts** are numbered RCPT/<financial year>/NNNN, rendered as a PDF and can be emailed to the client. Deleting a payment recalculates the invoice.
- **Online payments** (business switch, needs Razorpay keys): the public invoice page shows **Pay online**; the server creates a Razorpay order, Razorpay's hosted checkout handles UPI, cards, net banking and wallets, and the signature is verified on return. A webhook settles the payment too, so a closed browser tab still records it. Orders and their status live in a payment_orders table and are never recorded twice.
- Every recorded payment adds a "Payment received" notification, and a fully settled invoice adds "Paid in full" and stops all reminders for it.
- The Payments page lists receipts with search, method and date filters, totals in INR and CSV export.

Email, automatic reminders and notify mail

All email goes through one provider layer: SMTP when SMTP_HOST is set, otherwise Resend, otherwise a console fallback that prints the message to the server log. Every client email is written to an audit table with status, provider and message id.

Emails the app sends

Email	To	When
Invoice / quote / credit note, PDF attached	Client	On demand, or automatically on issue when "Auto email invoices" is on
Payment reminder, PDF attached	Client	On the schedule below, or by hand from the invoice or client page
Client statement, all unpaid PDFs attached	Client	By hand from the clients list or client page
Receipt, PDF attached	Client	On demand after recording a payment
Notify mail: what was sent and what is pending	Business users	After any run that sent reminders, when "Notify me" is on
Password reset link	User	On request

Reminder schedule (per business, Settings, Payment reminders; defaults shown)

Step	Default	Sent
Before the due date	7, 3 and 1 days before	One email per step
On the due date	always	One email
After the due date (pending payment)	every 7 days	One email per period while unpaid

Each step is recorded against the invoice, so re-running never duplicates an email, and a missed day sends only the latest step. A reminder sent by hand the same day counts as that day's step. Clients can be opted out individually; manual sends ignore the opt-out. Reminder emails state the amount due, the due date, how many days remain or are overdue, and include the bank details from Settings.

Notifications, dashboard, reports and exports

Everything that happens to money shows up in one feed, and the dashboard and reports summarise it in INR.

- **Notifications**: a page in the sidebar and a bell in the header with an unread badge that opens a right-hand panel. Cards are grouped by day, newest first, and tinted by severity (info, success, warning, urgent). Kinds: invoice issued, invoice emailed, payment received, paid in full, due soon, due today, overdue, cancelled, reminder sent, reminder failed, statement sent. Clicking a card marks it read and opens the record; "Mark all as read" and "Check due invoices" act on the whole feed.
- **Dashboard**: total invoiced, paid, pending and overdue, a 12-month invoiced-versus-collected chart, invoice status breakdown, recent invoices and payments.
- **Reports**: for a chosen date range, invoiced, collected (with collection rate), outstanding and overdue, tax billed and average invoice; revenue by client, by currency, by status and by month.
- **History**: one timeline of every invoice issued and payment received, filterable by type, client and date.
- **Records**: every client, invoice and payment in one place.
- **Search**: the header search finds invoices or clients as you type; the clients, invoices, payments and history pages have the same type-ahead on their own search boxes.
- **CSV export** of clients, invoices, payments and history.

Settings

Settings are per business and apply to new documents only; issued documents keep the details they were created with.

Section	What it controls
Business profile	Name, legal name, billing email (reply-to on emails), phone, website, PAN, logo
Address	Address lines, city, postal code, country, Indian state (sets your GST state)
Tax (GST)	GST on or off, GSTIN

Section	What it controls
Invoice defaults	Prefixes for invoices, quotes and credit notes; default currency; payment terms in days; quote validity in days; bank or UPI details printed on documents and emails; default notes and terms; round totals to the rupee; authorised signatory name and signature image
Sending, payments and reminders	Online payments via Razorpay; auto-email invoices on issue; automatic reminders on or off; days before due date (e.g. 7, 3, 1); overdue repeat interval; notify mail to business users
Appearance	7 accent themes and light, dark or system mode, from the account menu
Account	Your name, email and password

Automation and scheduling

One daily check does all the time-based work for a business, in this order:

1. Mark unpaid invoices past their due date as overdue.
2. Raise in-app notices for invoices due soon (within the largest schedule step), due today and overdue.
3. Send the reminder emails the schedule owes today and record each step.
4. Generate any recurring invoices that are due.
5. Email the notify mail to the business's users if anything was sent.

It runs three ways: automatically after a dashboard page loads, at most once an hour per business; on demand from **Check due invoices** on the Notifications page; and from the cron route `/api/cron/due-reminders`, which runs it for every business when called with the `CRON_SECRET` bearer token. The route is what a daily scheduler should call (Render Cron Job, Vercel Cron or any external cron). All steps are idempotent, so overlapping or repeated runs are safe.

Database schema

16 tables in PostgreSQL, every domain table carrying a business id; money columns are numeric(14,2), never floats.

Table	Purpose
businesses	Tenant profile, GST, numbering prefixes, defaults, payment and reminder settings
users, sessions, password_reset_tokens	Accounts, server-side sessions, single-use reset tokens
clients	Billing contacts with GST state, currency and reminder opt-out
items	Product and service catalog with HSN/SAC, unit, price and tax rate
invoices, invoice_items	Invoices, quotes and credit notes (document kind), their snapshots of GST and exchange rate, line items
document_counters	Per-business, per-kind, per-financial-year sequence behind every document number
recurring_profiles	Templates that generate invoices on a weekly to yearly cadence
payments, payment_orders	Recorded payments with receipt numbers; Razorpay orders and their status
email_logs	Audit of every client email: recipient, subject, provider, status, error
notifications	The in-app feed, with a dedupe key for scheduled notices
invoice_reminders	Ledger of automatic reminders by invoice and schedule step
exchange_rates	Cached rates to INR with their source and age

Migrations live in db/migrations (8 so far) and are applied with `npm run db:migrate`; the schema is in db/schema with one file per table.

Deployment and configuration

The app runs as one Node web service; it needs a hosted PostgreSQL database, an SMTP account and, optionally, Razorpay and Turnstile keys.

Variable	Required	Purpose
DATABASE_URL	yes	PostgreSQL connection string (Supabase transaction pooler in production)
NEXT_PUBLIC_APP_URL	yes	Public site URL used in email links and the public invoice page
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, SMTP_SECURE	yes for real email	SMTP relay; without a host the app falls back to Resend, then to the console
EMAIL_FROM (or SMTP_FROM)	yes	Sender address shown on every email
RESEND_API_KEY	optional	Alternative provider when SMTP is not set
CRON_SECRET	yes in production	Bearer token the scheduler sends to /api/cron/due-reminders
NEXT_PUBLIC_TURNSTILE_SITE_KEY, TURNSTILE_SECRET_KEY	optional	Cloudflare bot check on login; the site's hostname must be allowed on the widget
RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET	optional	Online payments; webhook URL is /api/webhooks/razorpay
EXCHANGE_RATE_CACHE_HOURS	optional	How long fetched rates are reused (default 12)

Deploy steps

1. Create the database and run `DATABASE_URL="<hosted string>" npm run db:migrate` once from a developer machine.
2. Set the variables above on the host and deploy from the GitHub main branch (`npm run build` , `npm run start`).
3. Schedule a daily call to the cron route, for example a Render Cron Job at 03:30 UTC running `curl -fsS -H "Authorization: Bearer $CRON_SECRET" https://<site>/api/cron/due-reminders` .
4. Add the production hostname to the Turnstile widget and the webhook URL to the Razorpay dashboard.

5. Add SPF and DKIM records for the sender domain so reminders reach inboxes.

Health check: `GET /api/health` reports, without secrets, whether the database answers, how many migrations and users it has, which email provider is active and whether Turnstile and the cron secret are configured. Use it first when something looks wrong on a deployment.

Known gaps and roadmap

The product is usable end to end today; these are the limits to know about and the natural next steps.

- No automated test suite yet; verification is type checking, lint and manual runs against the database.
- Scheduled work depends on an external daily cron call in production; without it, reminders and recurring invoices only run when someone opens the dashboard.
- All users of a business have the same rights; there are no roles or permissions.
- Razorpay is the only payment gateway, and it needs a verified Razorpay account.
- Email deliverability depends on SPF and DKIM for the sender domain; a new domain may land in spam at first.
- Dates use the server's local calendar day; businesses in other time zones could see a reminder a day early or late.
- Candidates for the next version: roles and permissions, a second gateway (Stripe) for foreign-currency clients, WhatsApp or SMS reminders, e-invoicing (IRN) and GST return exports, and automated tests around numbering, tax and the reminder schedule.